ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ

BÁO CÁO
ĐỒ ÁN THIẾT KẾ II

ĐỀ TÀI:

THIẾT KẾ HỆ THỐNG ĐO NHIỆT ĐỘ VÀ CẢNH BÁO NGƯỠNG SỬ DỤNG ATMEGA16

Giảng viên hướng dẫn:	ThS. Vũ Hồng Vinh	
Sinh viên thực hiện:	Mai Duy Hiếu	20223967
	Hoàng Quốc Hưng	20233758
Mã lớp học:	772642	
Nhóm thực hiện:	Nhóm 4	

Hà Nội, 11/2026

LỜI NÓI ĐẦU

Đề tài "Thiết kế hệ thống đo nhiệt độ và cảnh báo ngưỡng sử dụng ATmega16" thuộc học phần Đồ án Thiết kế II. Hệ thống đọc cảm biến DHT11, hiển thị dữ liệu lên LCD 1602 và bật 5 LED khi giá trị nhiệt độ nhận được đạt hoặc vượt ngưỡng cài đặt.

Nhóm vẽ lại sơ đồ nguyên lý từ tài liệu giảng viên cung cấp và thiết kế PCB trên cơ sở sơ đồ đó. Phần mềm gồm giao tiếp DHT11, cấu hình xung nhịp, vô hiệu hóa JTAG và điều khiển LCD, LED trên các cổng dùng chung.

Ngưỡng 30 °C được chọn làm mốc kiểm thử chức năng của mô hình; kết quả thực nghiệm tập trung vào hiển thị, cảnh báo ngưỡng và xử lý mất tín hiệu cảm biến.

Báo cáo gồm 5 chương:

- **Chương 1**: Tổng quan đề tài: đặt vấn đề, mục tiêu, phạm vi, yêu cầu thiết kế và lựa chọn linh kiện.
- **Chương 2**: Cơ sở lý thuyết về ATmega16, kit AVR, cảm biến DHT11, LCD 1602 và khối LED.
- **Chương 3**: Thiết kế sơ đồ khối, sơ đồ nguyên lý, PCB và kết nối phần cứng; lưu đồ và các thuật toán điều khiển; quy trình biên dịch và nạp firmware.
- **Chương 4**: Kết quả thực nghiệm và giới hạn kiểm chứng.
- **Chương 5**: Kết luận, hạn chế và hướng phát triển.

Chúng em trân trọng cảm ơn thầy ThS. Vũ Hồng Vinh đã hướng dẫn thực hiện đồ án, cùng các thầy cô Trường Điện – Điện tử đã giảng dạy những kiến thức nền tảng. Nhóm mong nhận được góp ý để tiếp tục hoàn thiện mô hình và báo cáo.

Hà Nội, tháng 11 năm 2026

Sinh viên thực hiện

Mai Duy Hiếu

Hoàng Quốc Hưng

MỤC LỤC

LỜI NÓI ĐẦU **i**

DANH MỤC HÌNH VẼ **v**

DANH MỤC BẢNG **vi**

DANH MỤC TỪ VIẾT TẮT **vii**

1 TỔNG QUAN ĐỀ TÀI **1**

1.1 Đặt vấn đề 1

1.2 Mục tiêu và phạm vi đề tài 1

1.2.1 Mục tiêu 1

1.2.2 Phạm vi 2

1.3 Yêu cầu thiết kế 2

1.4 Lựa chọn linh kiện 3

1.4.1 Lựa chọn cảm biến nhiệt độ 3

1.4.2 Lựa chọn thiết bị hiển thị và cảnh báo 4

1.5 Công cụ thực hiện 4

2 CƠ SỞ LÝ THUYẾT VÀ LINH KIỆN **5**

2.1 Vi điều khiển ATmega16 5

2.1.1 Tổng quan kiến trúc và đặc tính kỹ thuật 5

2.1.2 Cấu trúc các cổng vào/ra và chức năng trong đồ án 6

2.1.3 Xung nhịp hệ thống và xung đột với giao diện JTAG 6

2.2 Mạch kit thực hành vi điều khiển họ AVR 8

2.3 Cảm biến số nhiệt độ và độ ẩm DHT11 9

2.3.1 Cấu tạo và nguyên lý hoạt động 9

2.3.2 Giao thức truyền thông Single-Bus của DHT11 10

2.4 Màn hình hiển thị tinh thể lỏng LCD 1602 12
2.5 Khối cảnh báo LED 13

3 THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG 14

3.1 Thiết kế kiến trúc phần cứng và sơ đồ nguyên lý 14
3.1.1 Sơ đồ khối chức năng và kết nối liên khối hệ thống 14
3.1.2 Bảng kết nối dây chi tiết giữa các linh kiện 16
3.2 Xây dựng thuật toán và phần mềm điều khiển 16
3.2.1 Thuật toán điều khiển tổng quát chương trình 16
3.2.2 Thuật toán giao tiếp và giải mã cảm biến DHT11 19
3.2.3 Xây dựng thư viện điều khiển màn hình LCD 1602 20
3.2.4 Thuật toán so sánh ngưỡng cảnh báo và điều khiển LED 21
3.3 Quy trình biên dịch và nạp firmware 22
3.3.1 Quy trình tự động hóa với Makefile 22
3.3.2 Quy trình nạp chương trình bằng phần mềm ProgISP 23

4 KẾT QUẢ THỰC NGHIỆM 25

4.1 Kết quả thực nghiệm trên phần cứng 25
4.1.1 Giai đoạn khởi động 25
4.1.2 Trạng thái bình thường ($T < 30\,^{\circ}\mathrm{C}$) 25
4.1.3 Trạng thái cảnh báo quá nhiệt ($T \geq 30\,^{\circ}\mathrm{C}$) 26
4.1.4 Kiểm tra xử lý lỗi cảm biến 27
4.2 Đánh giá kết quả và phạm vi kiểm chứng 28
4.2.1 Kết quả chức năng đã ghi nhận 28
4.2.2 Giới hạn của kết quả thực nghiệm 28

5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 29

5.1 Kết luận 29
5.2 Hạn chế của đề tài 29

5.3 Hướng phát triển 30

TÀI LIỆU THAM KHẢO **31**

PHỤ LỤC: TOÀN BỘ MÃ NGUỒN CHƯƠNG TRÌNH **32**

DANH MỤC HÌNH VẼ

2.1 Sơ đồ phân bố chân của vi điều khiển ATmega16 dạng đóng vỏ DIP-40 5

2.2 Mô hình 3D kit AVR trong tài liệu giảng viên cung cấp 8

3.1 Sơ đồ kết nối liên khối chi tiết của hệ thống đo nhiệt độ và cảnh báo . . 14

3.2 Sơ đồ nguyên lý kit AVR được nhóm vẽ lại từ tài liệu giảng viên cung cấp 15

3.3 Mô hình 3D PCB do nhóm thiết kế trên Altium Designer 17

3.4 Lưu đồ thuật toán đo nhiệt độ, cảnh báo và xử lý lỗi 18

3.5 Giao diện ProgISP khi nạp firmware cho mô hình (fuse `0x99E4`) 24

4.1 Hình ảnh thực tế hệ thống trong các kịch bản thử nghiệm 26

DANH MỤC BẢNG

1.1 Yêu cầu thiết kế của hệ thống 2

1.2 So sánh các phương án cảm biến nhiệt độ 3

1.3 Phần cứng và phần mềm sử dụng trong đề tài 4

2.1 Thông số kỹ thuật chính của vi điều khiển ATmega16 6

2.2 Đặc tính kỹ thuật của cảm biến số DHT11 10

2.3 Cấu trúc gói dữ liệu 40 bit của DHT11 11

2.4 Kết nối chân giữa giắc J7 trên kit, LCD 1602 và ATmega16 12

3.1 Bảng tổng hợp kết nối tín hiệu phần cứng của hệ thống 16

3.2 Thời gian timeout của các vòng chờ trong `DHT11_Read` ở 8 MHz . . . 19

3.3 Dung lượng bộ nhớ của chương trình (`avr-size`, avr-gcc 16.1.0, `-Os`) 23

4.1 Tổng hợp các kịch bản kiểm tra và kết quả 27

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Ý nghĩa
ADC	Analog-to-Digital Converter – Bộ chuyển đổi tương tự – số
AVR	Họ vi điều khiển 8-bit của Atmel (nay thuộc Microchip)
DIP	Dual In-line Package – Kiểu đóng vỏ hai hàng chân cắm
EEPROM	Electrically Erasable Programmable Read-Only Memory
GPIO	General-Purpose Input/Output – Chân vào/ra đa dụng
I/O	Input/Output – Vào/ra
ISP	In-System Programming – Nạp chương trình trên mạch
JTAG	Joint Test Action Group – Chuẩn giao diện kiểm tra, gỡ lỗi (IEEE 1149.1)
LCD	Liquid Crystal Display – Màn hình tinh thể lỏng
LED	Light-Emitting Diode – Điốt phát quang
MCU	Microcontroller Unit – Vi điều khiển
NTC	Negative Temperature Coefficient – Hệ số nhiệt điện trở âm
PCB	Printed Circuit Board – Mạch in
PWM	Pulse Width Modulation – Điều chế độ rộng xung
RC	Resistor–Capacitor – Mạch dao động điện trở – tụ điện
RH	Relative Humidity – Độ ẩm tương đối
RISC	Reduced Instruction Set Computer – Kiến trúc tập lệnh rút gọn
SPI	Serial Peripheral Interface – Giao tiếp ngoại vi nối tiếp
SRAM	Static Random-Access Memory – Bộ nhớ truy cập ngẫu nhiên tĩnh
USART	Universal Synchronous/Asynchronous Receiver-Transmitter
USB	Universal Serial Bus

CHƯƠNG 1

TỔNG QUAN ĐỀ TÀI

1.1 Đặt vấn đề

Nhiệt độ là đại lượng cần được theo dõi trong nhiều môi trường như phòng làm việc, tủ điện, phòng đặt thiết bị hoặc kho bảo quản. Khi nhiệt độ vượt mức cho phép, thiết bị điện tử có thể hoạt động không ổn định hoặc giảm tuổi thọ, còn vật tư bảo quản có thể bị hư hỏng. Một hệ thống đo liên tục, hiển thị giá trị tại chỗ và báo hiệu ngay khi vượt ngưỡng giúp người vận hành phát hiện sự cố sớm mà không cần kiểm tra thủ công.

Với yêu cầu cơ bản gồm đo, hiển thị, so sánh ngưỡng và cảnh báo, một vi điều khiển 8 bit kết hợp cảm biến số và màn hình ký tự là đủ đáp ứng, với chi phí thấp và cấu trúc đơn giản. Trong học phần Đồ án Thiết kế II, nhóm thực hiện đề tài *"Thiết kế hệ thống đo nhiệt độ và cảnh báo ngưỡng sử dụng ATmega16"* trên kit AVR của học phần, qua đó vận dụng kiến thức về lập trình thanh ghi, giao tiếp ngoại vi, cấu hình xung nhịp và quy trình nạp chương trình cho vi điều khiển.

1.2 Mục tiêu và phạm vi đề tài

1.2.1 Mục tiêu

Đề tài hướng tới các mục tiêu sau:

- Xây dựng hệ thống đọc nhiệt độ từ cảm biến DHT11, hiển thị giá trị lên LCD 1602 và bật 5 LED cảnh báo khi nhiệt độ đạt hoặc vượt ngưỡng 30 °C.
- Xây dựng chương trình giao tiếp với DHT11 bằng thao tác thanh ghi, có cơ chế timeout để vi điều khiển không bị treo và kiểm tra checksum để loại bỏ gói tin sai.
- Vẽ lại sơ đồ nguyên lý kit AVR từ tài liệu giảng viên cung cấp và thiết kế PCB trên Altium Designer.
- Kiểm thử trên phần cứng các trạng thái hoạt động chính: nhiệt độ bình thường, vượt ngưỡng và mất tín hiệu cảm biến.

1.2.2 Phạm vi

Đề tài giới hạn trong các nội dung sau:

- Hệ thống dùng trong nhà, với nhiệt độ không âm. Ngưỡng 30 °C được cố định trong chương trình và dùng làm mốc kiểm thử chức năng, không phải ngưỡng an toàn của một ứng dụng cụ thể.
- DHT11 trả về cả nhiệt độ và độ ẩm, nhưng chương trình hiện chỉ hiển thị và cảnh báo theo nhiệt độ.
- Đề tài không bao gồm lưu trữ dữ liệu, truyền dữ liệu từ xa và cài đặt ngưỡng bằng phím bấm.
- Kết quả được đánh giá ở mức chức năng; việc đo độ chính xác so với nhiệt kế tham chiếu nằm ngoài phạm vi các thử nghiệm đã thực hiện (mục 4.2.2).

1.3 Yêu cầu thiết kế

Từ mục tiêu trên và điều kiện thực hiện trên kit AVR của học phần, nhóm đặt ra các yêu cầu thiết kế trong Bảng 1.1. Vi điều khiển ATmega16 có sẵn trên kit nên không thuộc phạm vi lựa chọn; các mục tiếp theo trình bày việc chọn cảm biến và thiết bị hiển thị, cảnh báo.

Bảng 1.1: Yêu cầu thiết kế của hệ thống

Nhóm yêu cầu	Chỉ tiêu	Cách đáp ứng
Đo nhiệt độ	Đo nhiệt độ trong phòng, bao gồm vùng quanh ngưỡng 30 °C; cập nhật khoảng 2 giây một lần	DHT11 (mục 1.4.1); chu kỳ đọc (mục 3.2.4)
Hiển thị	Hiện giá trị đo, ngưỡng và thông báo lỗi bằng chữ ngay trên mô hình	LCD 1602 (mục 1.4.2)
Cảnh báo	Báo khi nhiệt độ đạt hoặc vượt 30 °C bằng tín hiệu nhìn thấy được; tự trở về khi nhiệt độ giảm	5 LED trên PD0–PD4 (mục 3.2.4)
Độ tin cậy	Phát hiện mất tín hiệu và gói tin sai; vi điều khiển không bị treo khi cảm biến hỏng	Timeout và checksum (mục 3.2.2)
Phần cứng	Dùng kit ATmega16 có sẵn, nguồn 5 V, nạp qua USBasp; hạn chế linh kiện ngoài kit	Kết nối ở Bảng 3.1

1.4 Lựa chọn linh kiện

1.4.1 Lựa chọn cảm biến nhiệt độ

Bảng 1.2 so sánh ba cảm biến nhiệt độ phổ biến khi làm việc với vi điều khiển AVR [2, 3, 4].

Bảng 1.2: So sánh các phương án cảm biến nhiệt độ

Tiêu chí	LM35	DS18B20	DHT11
Tín hiệu ra	Điện áp tương tự, 10 mV/°C	Số, bus 1-Wire (Dallas)	Số, Single-Bus (Aosong)
Dải đo nhiệt độ	−55 đến 150 °C	−55 đến 125 °C	0 đến 50 °C
Độ chính xác	0,5 °C tại 25 °C	±0,5 °C trong khoảng −10 đến 85 °C	±2 °C
Độ phân giải	Phụ thuộc ADC: khoảng 0,49 °C với ADC 10 bit, tham chiếu 5 V	0,5 đến 0,0625 °C (9–12 bit)	1 °C theo tài liệu tham chiếu
Nguồn cấp	4–30 V	3,0–5,5 V	3,0–5,5 V
Thời gian đọc	Một lần chuyển đổi ADC	Tối đa 750 ms ở 12 bit	Khoảng 25 ms; hai lần đọc cách nhau tối thiểu 1 s
Đo độ ẩm	Không	Không	Có (20–90 % RH)
Kết nối với MCU	Một kênh ADC; dây tín hiệu tương tự dễ nhiễm nhiễu	Một chân GPIO, điện trở kéo lên	Một chân GPIO, điện trở kéo lên

LM35 và DS18B20 chính xác hơn DHT11. Nhóm vẫn chọn DHT11 vì các lý do sau:

- Dải 0–50 °C và sai số ±2 °C đáp ứng yêu cầu cảnh báo ngưỡng 30 °C trong phòng ở Bảng 1.1.
- Tín hiệu số chỉ cần một chân GPIO (PA1); không chiếm kênh ADC và không phụ thuộc điện áp tham chiếu hay nhiễu trên dây như LM35.
- Module 3 chân dễ đấu nối ngoài kit; cảm biến đo thêm độ ẩm, cho phép mở rộng

chức năng mà không đổi phần cứng.

- Giao thức buộc chương trình tự tạo và lấy mẫu xung theo thời gian, phù hợp với nội dung lập trình thanh ghi của học phần.

Đổi lại, sai số của DHT11 lớn hơn hai phương án còn lại. Nếu ứng dụng cần độ chính xác cao hơn, có thể thay bằng DS18B20 trên cùng chân PA1; thay đổi chủ yếu nằm ở phần mềm giao tiếp.

1.4.2 Lựa chọn thiết bị hiển thị và cảnh báo

Kit có sẵn ba thiết bị đầu ra dùng được: LED 7 thanh một chữ số D11, dãy 8 LED đơn trên PORT D và giắc J7 dành cho LCD 1602 (Hình 3.2). LED 7 thanh một chữ số không đủ để hiện giá trị như `26.5` hay các thông báo lỗi, đồng thời dùng chung PC0–PC7 với giắc LCD. Nhóm chọn LCD 1602 (2 dòng × 16 ký tự, bộ điều khiển HD44780) vì đủ chỗ cho giá trị đo, ngưỡng và thông báo chẩn đoán bằng chữ; LCD chạy ở chế độ 4 bit để chỉ chiếm PC4–PC7 và PD5–PD7.

Khối cảnh báo dùng 5 LED đơn trên PD0–PD4: các LED có sẵn trên kit, không cần mạch ngoài và dễ quan sát. Kit không có còi báo nên phương án này được đưa vào hướng phát triển (mục 5.3).

1.5 Công cụ thực hiện

Bảng 1.3 liệt kê phần cứng và phần mềm nhóm sử dụng.

Bảng 1.3: Phần cứng và phần mềm sử dụng trong đề tài

Thành phần	Mục đích sử dụng
Kit AVR ATmega16	Nền tảng phần cứng chính: nguồn 5 V, vi điều khiển, giắc LCD, dãy LED
Module DHT11, LCD 1602	Cảm biến nhiệt độ và thiết bị hiển thị
Mạch nạp USBasp	Nạp chương trình và fuse qua cổng ISP
Altium Designer	Vẽ lại sơ đồ nguyên lý và thiết kế PCB
avr-gcc, GNU Make	Biên dịch mã nguồn C thành tệp `.hex` (mục 3.3.1)
ProgISP 1.72	Nạp firmware và ghi fuse (mục 3.3.2)

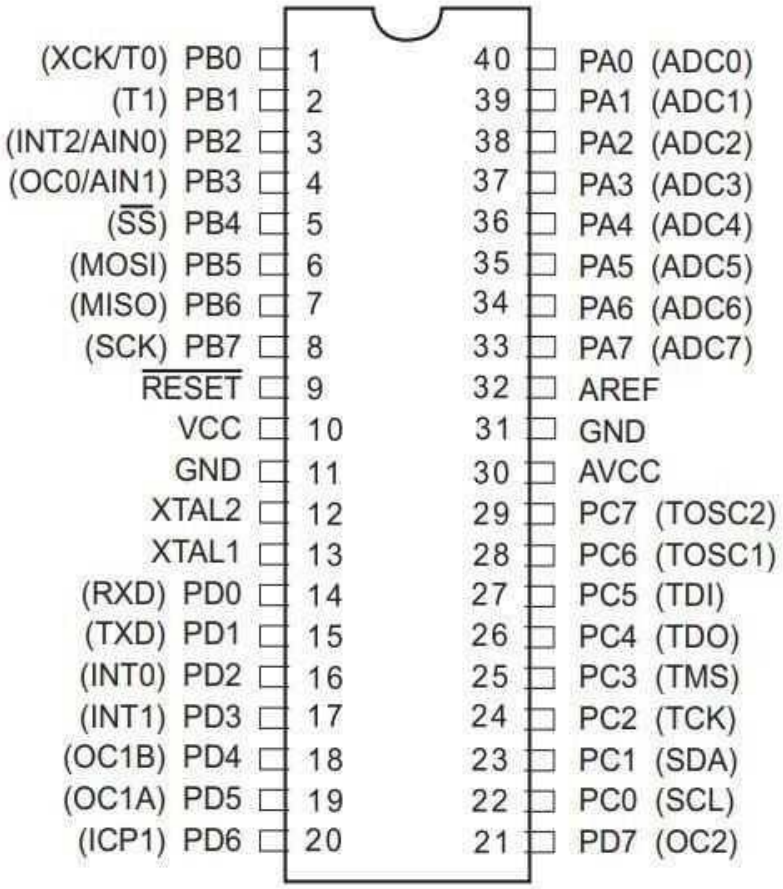
CHƯƠNG 2

CƠ SỞ LÝ THUYẾT VÀ LINH KIỆN

2.1 Vi điều khiển ATmega16

2.1.1 Tổng quan kiến trúc và đặc tính kỹ thuật

ATmega16 là vi điều khiển 8 bit họ AVR của Atmel (nay thuộc Microchip Technology), sử dụng kiến trúc RISC với 131 lệnh, phần lớn thực thi trong một chu kỳ clock [1]. Thông lượng xử lý có thể đạt gần 1 MIPS trên mỗi MHz, phù hợp với yêu cầu đọc cảm biến và điều khiển hiển thị của mô hình.



Hình 2.1: Sơ đồ phân bố chân của vi điều khiển ATmega16 dạng đóng vỏ DIP-40

Bảng 2.1 tổng hợp các thông số của ATmega16 liên quan đến đồ án.

Bảng 2.1: Thông số kỹ thuật chính của vi điều khiển ATmega16

Thông số	Giá trị
Kiến trúc CPU	AVR RISC 8 bit, 32 thanh ghi đa năng 8 bit
Bộ nhớ Flash chương trình	16 KB, nạp lại được trên mạch, độ bền 10.000 lần ghi/xóa
Bộ nhớ SRAM	1 KB (1024 byte)
Bộ nhớ EEPROM	512 byte, độ bền 100.000 lần ghi/xóa
Tần số xung nhịp	0–16 MHz (điện áp 4,5–5,5 V)
Số chân I/O	32 chân, chia thành 4 cổng 8 bit (PORT A, B, C, D)
Giao diện gỡ lỗi	JTAG (IEEE 1149.1), hỗ trợ On-Chip Debug
Điện áp hoạt động	4,5–5,5 V với ATmega16 (2,7–5,5 V với ATmega16L)

2.1.2 Cấu trúc các cổng vào/ra và chức năng trong đồ án

ATmega16 có 32 chân vào/ra hai chiều, chia đều cho 4 cổng từ A đến D. Mỗi cổng được điều khiển bởi 3 thanh ghi:

- DDRx (hướng dữ liệu): bit bằng 0 đặt chân làm đầu vào, bit bằng 1 đặt chân làm đầu ra.
- PORTx (dữ liệu xuất): khi chân là đầu ra, ghi 0 hoặc 1 sẽ xuất mức thấp (0 V) hoặc mức cao (5 V). Khi chân là đầu vào, ghi 1 sẽ bật điện trở kéo lên nội, ghi 0 đưa chân về trạng thái trở kháng cao.
- PINx (đọc dữ liệu vào): đọc mức logic thực tế tại các chân.

Trong đề tài, PA1 nối với chân dữ liệu của DHT11, PC4–PC7 và PD5–PD7 dùng cho LCD 1602, PD0–PD4 điều khiển 5 LED cảnh báo; bảng kết nối đầy đủ ở Bảng 3.1.

2.1.3 Xung nhịp hệ thống và xung đột với giao diện JTAG

Cấu hình nguồn xung nhịp 8 MHz bằng fuse

ATmega16 có sẵn mạch dao động RC nội đã hiệu chuẩn, tạo được các tần số 1, 2, 4 và 8 MHz mà không cần thạch anh hay tụ ngoài.

Sơ đồ kit có bố trí thạch anh ngoài `XT1` và hai tụ tải, nhưng đồ án chọn dao động RC nội 8 MHz nên không sử dụng khối này. Giá trị `F_CPU = 8000000UL` chỉ phục vụ

tính toán trễ khi biên dịch; tần số phần cứng do fuse quyết định. Fuse được ghi ở bước nạp chương trình bằng ProgISP (mục 3.3.2, Hình 3.5), trong đó:

- Low Fuse = `0xE4`: `CKSEL3..0 = 0100` chọn dao động RC nội 8 MHz; `SUT1..0 = 10` đặt thời gian khởi động 6 chu kỳ clock cộng trễ điển hình 65 ms ở 5 V (Bảng 10 của datasheet).

Xung đột giữa JTAG và LCD trên PORT C

Ở trạng thái xuất xưởng, fuse `JTAGEN` trên High Fuse đã được lập trình (`JTAGEN = 0`), tức giao diện JTAG được bật. Khi đó JTAG chiếm 4 chân của cổng C:

- PC2 = TCK (Test Clock)
- PC3 = TMS (Test Mode Select)
- PC4 = TDO (Test Data Output)
- PC5 = TDI (Test Data Input)

Bus dữ liệu 4 bit của LCD 1602 nối vào PC4–PC7. Khi JTAG còn bật, PC4 và PC5 do khối gỡ lỗi điều khiển và không hoạt động như GPIO, nên LCD không nhận được chuỗi lệnh khởi tạo. Triệu chứng thường gặp là dòng 1 hiện toàn ô đen, dòng 2 để trống.

Có hai cách tắt JTAG để dùng PC4, PC5 cho LCD:

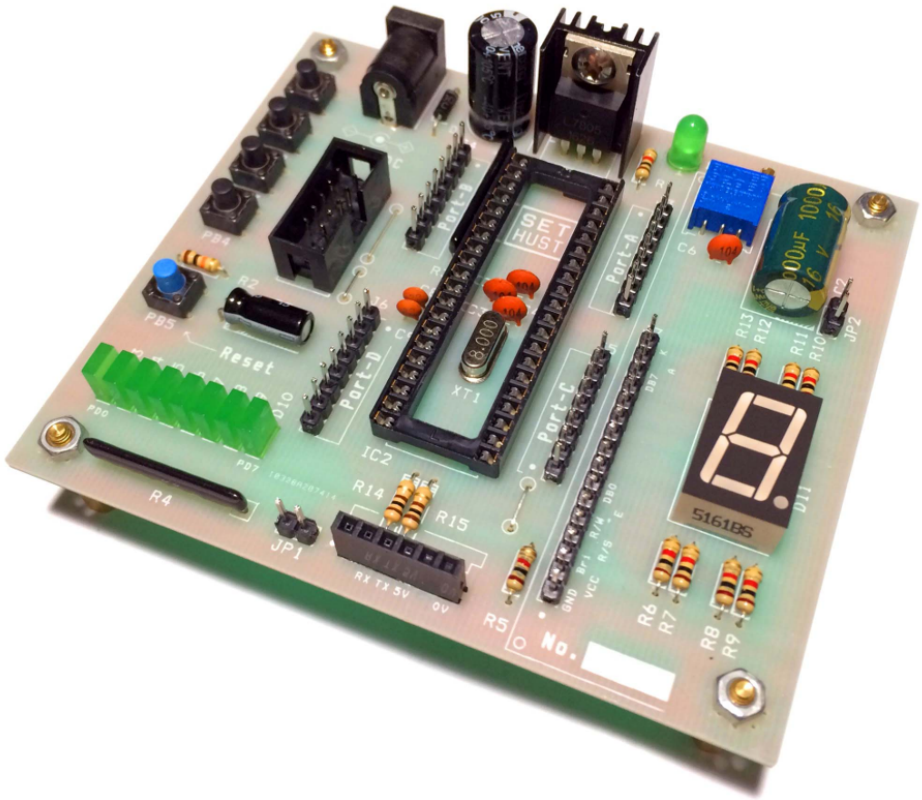
1. Bằng fuse: đặt High Fuse = `0xD9`. Khi đó bit 6 (`JTAGEN`) bằng 1 nên JTAG bị tắt, bit 5 (`SPIEN`) bằng 0 nên vẫn giữ cổng nạp SPI cho USBasp.
2. Bằng phần mềm: ghi bit `JTD` của thanh ghi `MCUCSR` hai lần liên tiếp trong vòng 4 chu kỳ máy, ngay đầu hàm `main()`:

```
MCUCSR = (1 << JTD);
MCUCSR = (1 << JTD);
```

Ở lần nạp chương trình ghi nhận trong Hình 3.5, fuse được ghi là `0x99E4`: Low Fuse `0xE4` chọn dao động RC nội 8 MHz như trên, còn High Fuse `0x99` giữ nguyên giá trị xuất xưởng, nghĩa là JTAG vẫn được bật ở mức fuse. Vì vậy trên mô hình, JTAG được tắt bằng cách thứ hai: lệnh ghi `JTD` ở đầu `main()` giải phóng PC4, PC5 trước khi khởi tạo LCD, và LCD hoạt động bình thường như kết quả ở Chương 4. Giá trị `0xD9` trong Makefile (mục 3.3.1) tắt JTAG thêm ở mức fuse, nên chương trình không phụ thuộc hoàn toàn vào lệnh phần mềm.

2.2 Mạch kit thực hành vi điều khiển họ AVR

Kit AVR sử dụng trong đồ án được xây dựng theo tài liệu do giảng viên cung cấp [6]; Hình 2.2 là mô hình bố trí linh kiện của kit trong tài liệu này. Dựa trên sơ đồ nguyên lý của kit, nhóm vẽ lại sơ đồ và thiết kế PCB riêng trên Altium Designer (Hình 3.2 và Hình 3.3).



Hình 2.2: Mô hình 3D kit AVR trong tài liệu giảng viên cung cấp [6]

Kit gồm các khối chức năng sau:

1. Đế cắm vi điều khiển DIP-40: cho phép lắp và thay vi điều khiển trong quá trình thực hành.
2. Khối nguồn 5 V: IC ổn áp tuyến tính LM7805 hạ áp từ adapter 9–12 V DC; kit cũng nhận nguồn 5 V trực tiếp từ cổng USB qua mạch nạp USBasp. Khối nguồn có tụ lọc và LED báo nguồn.
3. Cổng nạp ISP 10 chân (IDC-10): nối với USBasp qua các đường MOSI (PB5), MISO (PB6), SCK (PB7), RESET và GND.

4. Giắc J7 cho LCD 1602: hàng chân đơn 16 chân, tương thích màn hình dùng bộ điều khiển HD44780; biến trở VR1 dùng để chỉnh độ tương phản.

5. Dãy LED đơn và nút bấm: 8 LED đơn nối với PORT D và các nút bấm nối với PORT B, phục vụ bài thực hành vào/ra cơ bản. Đồ án dùng 5 LED trên PD0–PD4 để cảnh báo; 3 LED còn lại trên PD5–PD7 nằm chung đường với tín hiệu điều khiển LCD nên sáng theo mức logic của RW, RS, EN.

2.3 Cảm biến số nhiệt độ và độ ẩm DHT11

2.3.1 Cấu tạo và nguyên lý hoạt động

DHT11 là cảm biến số đo nhiệt độ và độ ẩm, bên trong có vi điều khiển 8 bit để số hóa và hiệu chuẩn dữ liệu trước khi truyền ra ngoài [2]. Cảm biến gồm hai phần tử cảm nhận:

- Phần tử đo độ ẩm kiểu điện trở: màng polymer nhạy ẩm có các điện cực xen kẽ. Khi độ ẩm thay đổi, lượng hơi nước hấp thụ vào màng thay đổi làm điện trở của màng thay đổi theo; vi điều khiển bên trong quy đổi điện trở này thành độ ẩm tương đối (%RH).
- Phần tử đo nhiệt độ NTC: điện trở nhiệt có hệ số nhiệt âm, điện trở giảm khi nhiệt độ tăng theo một quan hệ phi tuyến. Vi điều khiển bên trong quy đổi điện trở này thành nhiệt độ (°C).

Bảng 2.2: Đặc tính kỹ thuật của cảm biến số DHT11

Thông số kỹ thuật	Giá trị danh định
Điện áp hoạt động	3,0–5,5 V DC
Dải đo nhiệt độ	0–50 °C
Độ chính xác đo nhiệt độ	±2 °C
Độ phân giải nhiệt độ	1 °C
Dải đo độ ẩm	20–90 %RH
Độ chính xác đo độ ẩm	±5 %RH
Độ phân giải độ ẩm	1 %RH
Khoảng cách giữa các lần đọc	Tối thiểu 1 s; chương trình chọn trễ 2 s mỗi vòng lặp
Dòng điện tiêu thụ	0,5–2,5 mA khi đo; 100–150 μA khi chờ
Chuẩn giao tiếp	Single-Bus của Aosong (một dây dữ liệu)

Các giá trị trong Bảng 2.2 lấy theo tài liệu tham chiếu [2]. Module sử dụng trong mô hình là loại DHT11 gắn trên PCB, đầu nối 3 chân, mã bán hàng `LK_02463`; mã này không xác định phiên bản chip hay bản datasheet. Khi đấu nối cần dựa trên nhãn VCC, DATA (hoặc S) và GND trên module, không áp dụng trực tiếp số chân của cảm biến rời 4 chân. Sai khác giữa độ phân giải trong tài liệu và giá trị module trả về thực tế được phân tích ở mục 4.2.1.

2.3.2 Giao thức truyền thông Single-Bus của DHT11

Giao thức một dây (Single-Bus) của DHT11 do Aosong thiết kế là giao thức điểm–điểm riêng, **không tương thích** với chuẩn 1-Wire của Dallas/Maxim. Chuẩn 1-Wire dùng mã định danh ROM 64 bit, các khe thời gian do master điều khiển và cho phép nhiều thiết bị trên cùng bus. Ngược lại, với DHT11, vi điều khiển chỉ phát một xung khởi động, sau đó cảm biến tự phát liên tiếp 40 bit dữ liệu theo các mốc thời gian quy định.

DHT11 giao tiếp với vi điều khiển qua một chân DATA duy nhất. Đường này cần điện trở kéo lên nguồn 5 V (4,7–10 kΩ) để giữ mức cao khi bus rảnh.

Một lần đọc (40 bit) gồm các giai đoạn sau.

1. Tín hiệu khởi động

- Vi điều khiển đặt PA1 làm đầu ra và kéo bus xuống mức thấp tối thiểu 18 ms; chương trình chọn `_delay_ms(20)` để có dự phòng.

- Vi điều khiển nhả bus: chuyển PA1 sang đầu vào và bật điện trở kéo lên nội, để cảm biến có thể kéo bus xuống mà không bị vi điều khiển giữ ở mức cao. Bus trở về mức cao, rồi sau khoảng 20–40 μs DHT11 bắt đầu phản hồi.
- Chương trình chờ cố định 30 μs (`_delay_us(30)`) rồi mới vào vòng chờ phản hồi. Thời điểm này nằm trong khoảng 20–40 μs, nên bus có thể vẫn ở mức cao hoặc đã bị DHT11 kéo xuống. Vòng chờ đầu tiên (mã lỗi −1, Bảng 3.2) chỉ thoát khi bus ở mức thấp, nên cả hai trường hợp đều được xử lý đúng: nếu bus còn cao, chương trình chờ đến khi DHT11 kéo xuống; nếu bus đã thấp, vòng chờ kết thúc ngay.

2. Tín hiệu phản hồi từ cảm biến

Khi nhận được xung khởi động hợp lệ, DHT11 kéo bus xuống mức thấp khoảng 80 μs, rồi lên mức cao khoảng 80 μs để báo sẵn sàng truyền dữ liệu.

3. Truyền 40 bit dữ liệu

DHT11 phát lần lượt 40 bit (5 byte). Mỗi bit bắt đầu bằng mức thấp khoảng 50 μs; giá trị bit được mã hóa bằng độ rộng của mức cao tiếp theo:

- Bit ‘0’: mức cao kéo dài 26–28 μs.
- Bit ‘1’: mức cao kéo dài khoảng 70 μs.

Cách vi điều khiển phân biệt hai độ rộng xung này được trình bày ở mục 3.2.2.

4. Cấu trúc gói tin và kiểm tra lỗi

Gói dữ liệu gồm 5 byte, truyền theo thứ tự trong Bảng 2.3.

Bảng 2.3: Cấu trúc gói dữ liệu 40 bit của DHT11

Byte 0	Byte 1	Byte 2	Byte 3	Byte 4
RH_{int} Độ ẩm, phần nguyên	RH_{dec} Độ ẩm, phần thập phân	T_{int} Nhiệt độ, phần nguyên	T_{dec} Nhiệt độ, phần thập phân	Checksum Tổng kiểm tra

Byte checksum bằng 8 bit thấp của tổng 4 byte dữ liệu:

$$\text{Checksum} = (RH_{int} + RH_{dec} + T_{int} + T_{dec}) \bmod 256 \quad (2.1)$$

Gói tin được coi là hợp lệ nếu giá trị tính được trùng với Byte 4. Checksum 8 bit phát hiện được phần lớn lỗi do méo xung hoặc sai một bit, nhưng không sửa được lỗi và không phát hiện được trường hợp nhiều bit sai bù trừ nhau.

2.4 Màn hình hiển thị tinh thể lỏng LCD 1602

LCD 1602 là module hiển thị ký tự gồm 2 dòng, mỗi dòng tối đa 16 ký tự dạng ma trận điểm 5 × 8, dùng bộ điều khiển Hitachi HD44780 [5].

Bảng 2.4: Kết nối chân giữa giắc J7 trên kit, LCD 1602 và ATmega16

Chân J7	Chân LCD	Tên chân	**Chân MCU**	Chức năng
1	1	V_{SS}	**GND**	Cực âm nguồn (0 V)
2	2	V_{DD}	**VCC**	Cực dương nguồn (+5 V)
3	3	V_0	**VR1**	Điện áp chỉnh độ tương phản
4	4	RS	**PD6**	Chọn thanh ghi (0: lệnh; 1: dữ liệu)
5	5	RW	**PD5**	Chọn chiều (0: ghi vào LCD; 1: đọc từ LCD)
6	6	E	**PD7**	Xung chốt dữ liệu (tích cực sườn xuống)
7–10	7–10	D0–D3	**PC0–PC3**	4 bit dữ liệu thấp (không dùng ở chế độ 4 bit)
11	11	D4	**PC4**	Bit dữ liệu D4
12	12	D5	**PC5**	Bit dữ liệu D5
13	13	D6	**PC6**	Bit dữ liệu D6
14	14	D7	**PC7**	Bit dữ liệu D7
15	15	A (LED+)	**VCC**	Anot đèn nền
16	16	K (LED−)	**GND**	Katot đèn nền

Để tiết kiệm chân vi điều khiển, đồ án dùng chế độ giao tiếp 4 bit:

- Vi điều khiển chỉ dùng 4 chân PC4–PC7 nối tới D4–D7 của LCD, ít hơn 4 chân so với chế độ 8 bit.
- Mỗi byte lệnh hoặc dữ liệu được tách thành hai nửa: 4 bit cao truyền trước kèm một xung Enable trên PD7, sau đó 4 bit thấp được dịch lên và truyền kèm xung Enable thứ hai.

2.5 Khối cảnh báo LED

5 LED cảnh báo trên kit nối vào các chân PD0–PD4 theo cấu hình tích cực mức thấp (active low):

- Anot của các LED được cấp nguồn VCC qua mạng điện trở hạn dòng R4 và jumper JP1. Trên sơ đồ kit, R4 có giá trị 10 kΩ.
- Katot của các LED nối về các chân PD0–PD4.
- Khi chân xuất mức 0: dòng điện chạy từ VCC qua LED vào chân vi điều khiển, LED sáng (cảnh báo).
- Khi chân xuất mức 1: điện áp hai đầu LED xấp xỉ 0 V nên LED tắt (không cảnh báo).

CHƯƠNG 3

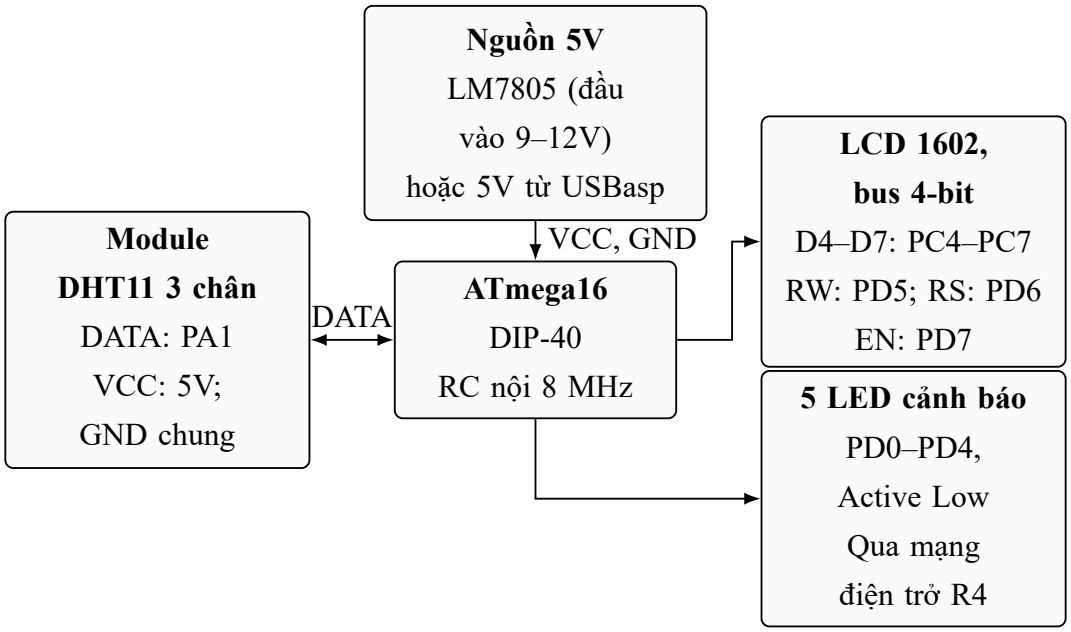
THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG

3.1 Thiết kế kiến trúc phần cứng và sơ đồ nguyên lý

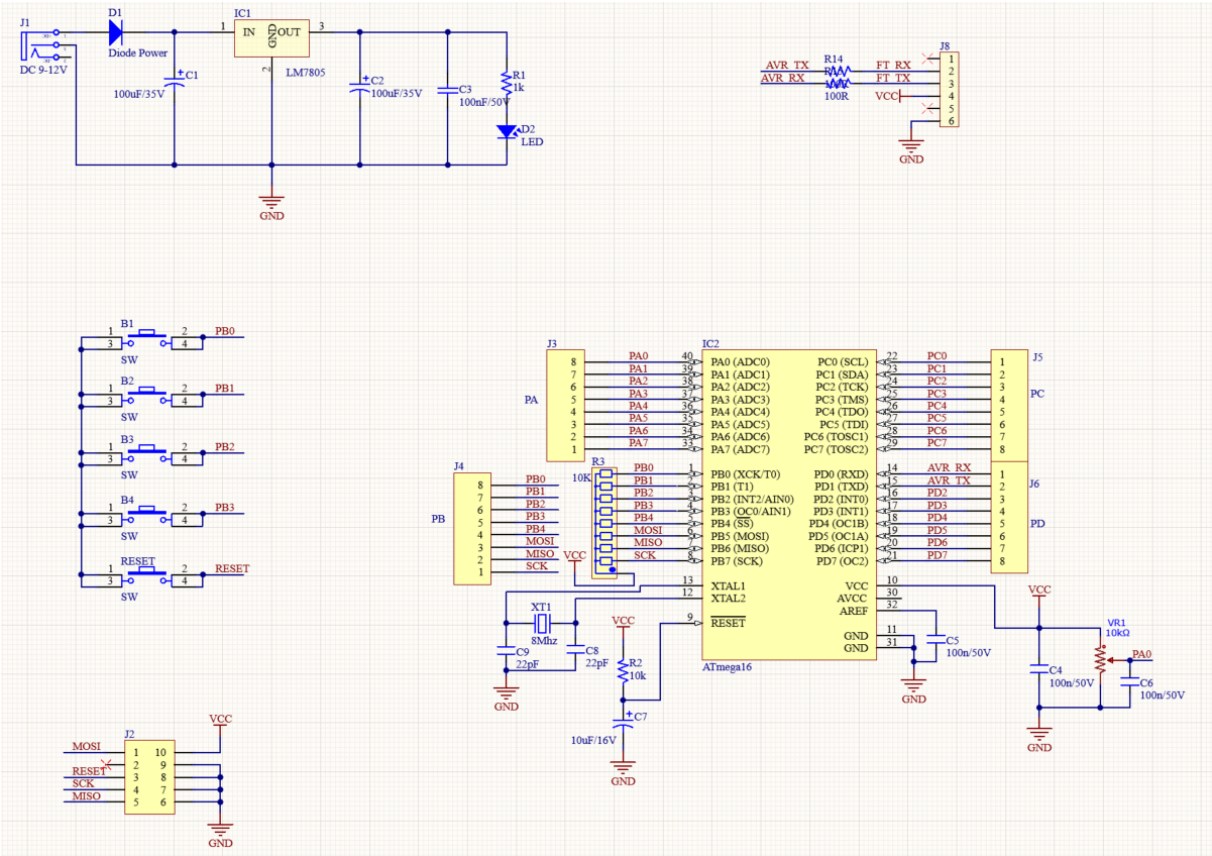
3.1.1 Sơ đồ khối chức năng và kết nối liên khối hệ thống

Nhóm vẽ lại sơ đồ giảng viên cung cấp (Hình 3.2) [6] và thiết kế PCB trên Altium Designer (Hình 3.3). DHT11 nối ngoài kit theo Hình 3.1. Hệ thống gồm 5 khối chức năng:

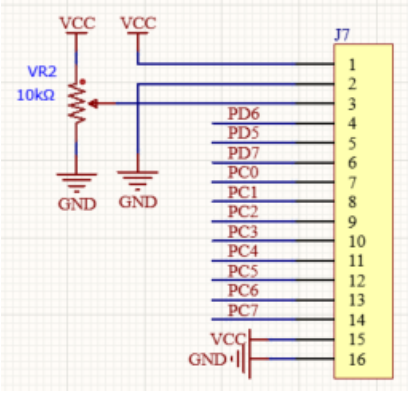
1. **Khối nguồn 5 V**: Cấp điện áp 5 V cho toàn mạch qua IC LM7805 hoặc từ cổng USB máy tính.
2. **Khối MCU trung tâm**: ATmega16 (DIP-40, dao động RC nội 8 MHz) điều khiển toàn bộ hoạt động của hệ thống.
3. **Khối cảm biến**: DHT11 đo nhiệt độ và độ ẩm, giao tiếp một dây với vi điều khiển qua chân PA1.
4. **Khối hiển thị**: LCD 1602 giao tiếp 4 bit qua các chân dữ liệu PC4–PC7 và chân điều khiển PD5–PD7.
5. **Khối cảnh báo**: 5 LED đơn nối với chân PD0–PD4, tích cực mức thấp qua mạng điện trở R4 trên kit.



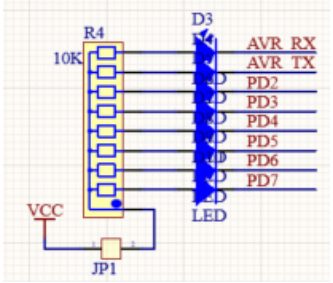
Hình 3.1: Sơ đồ kết nối liên khối chi tiết của hệ thống đo nhiệt độ và cảnh báo



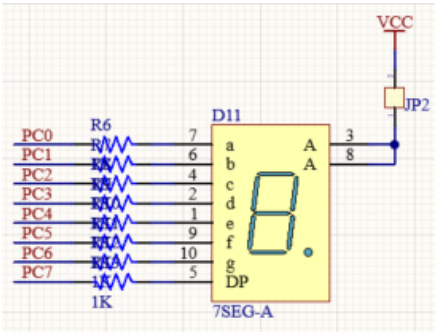
(a) Khối nguồn, vi điều khiển ATmega16, nút bấm, cổng nạp ISP (J2) và các giắc mở rộng cổng



(b) Giắc J7 nối LCD 1602



(c) Dãy LED đơn



(d) LED 7 thanh

Hình 3.2: Sơ đồ nguyên lý kit AVR được nhóm vẽ lại từ tài liệu giảng viên cung cấp

Ghi chú: Các khối (a)–(d) nối với nhau qua nhãn mạng (PA0–PD7, VCC, GND). Trong đồ án, LCD dùng PC4–PC7 và PD5–PD7; 5 LED cảnh báo dùng PD0–PD4. Khối thạch anh XT1 có trên sơ đồ nhưng chương trình chọn dao động RC nội 8 MHz. Cảm biến DHT11 là module nối ngoài, không nằm trong sơ đồ kit này.

Bảng 3.1: Bảng tổng hợp kết nối tín hiệu phần cứng của hệ thống

Khối chức năng	Chân linh kiện	Chân MCU	**Tên tín hiệu MCU**	Chức năng kỹ thuật
Cảm biến DHT11	DATA / S	Chân 39	**PA1 / ADC1**	Giao tiếp Single-Bus hai chiều
	VCC	–	**+5 V của kit**	Nguồn nuôi 5 V DC
	GND	–	**GND của kit**	Nối đất chung
Màn hình LCD	RS (Chân 4)	Chân 20	**PD6 / ICP1**	Chọn thanh ghi (0: lệnh, 1: dữ liệu)
	RW (Chân 5)	Chân 19	**PD5 / OC1A**	Chọn đọc/ghi (0: ghi)
	EN (Chân 6)	Chân 21	**PD7 / OC2**	Xung kích hoạt chốt dữ liệu
	D4 (Chân 11)	Chân 26	**PC4 / TDO**	Đường truyền dữ liệu bit 4
	D5 (Chân 12)	Chân 27	**PC5 / TDI**	Đường truyền dữ liệu bit 5
	D6 (Chân 13)	Chân 28	**PC6 / TOSC1**	Đường truyền dữ liệu bit 6
	D7 (Chân 14)	Chân 29	**PC7 / TOSC2**	Đường truyền dữ liệu bit 7
Dãy LED cảnh báo	Katot LED 1	Chân 14	**PD0 / RXD**	Cảnh báo, sáng khi mức 0
	Katot LED 2	Chân 15	**PD1 / TXD**	Cảnh báo, sáng khi mức 0
	Katot LED 3	Chân 16	**PD2 / INT0**	Cảnh báo, sáng khi mức 0
	Katot LED 4	Chân 17	**PD3 / INT1**	Cảnh báo, sáng khi mức 0
	Katot LED 5	Chân 18	**PD4 / OC1B**	Cảnh báo, sáng khi mức 0

3.1.2 Bảng kết nối dây chi tiết giữa các linh kiện

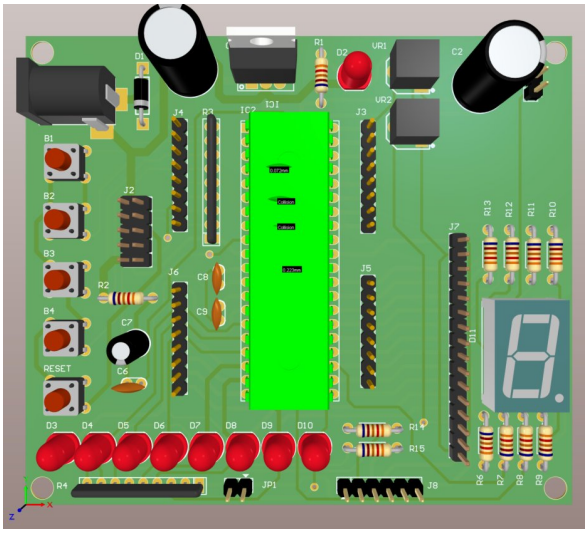
Bảng 3.1 liệt kê kết nối giữa từng khối chức năng và chân của ATmega16; số chân tính theo vỏ DIP-40 (Hình 2.1). Tên tín hiệu sau dấu "/" là chức năng thay thế của chân, không được dùng trong đồ án.

3.2 Xây dựng thuật toán và phần mềm điều khiển

3.2.1 Thuật toán điều khiển tổng quát chương trình

Chương trình vận hành tuần tự theo các bước dưới đây. Lưu đồ ở Hình 3.4 thể hiện cùng các nhánh xử lý; mã nguồn đầy đủ nằm ở Phụ lục A.

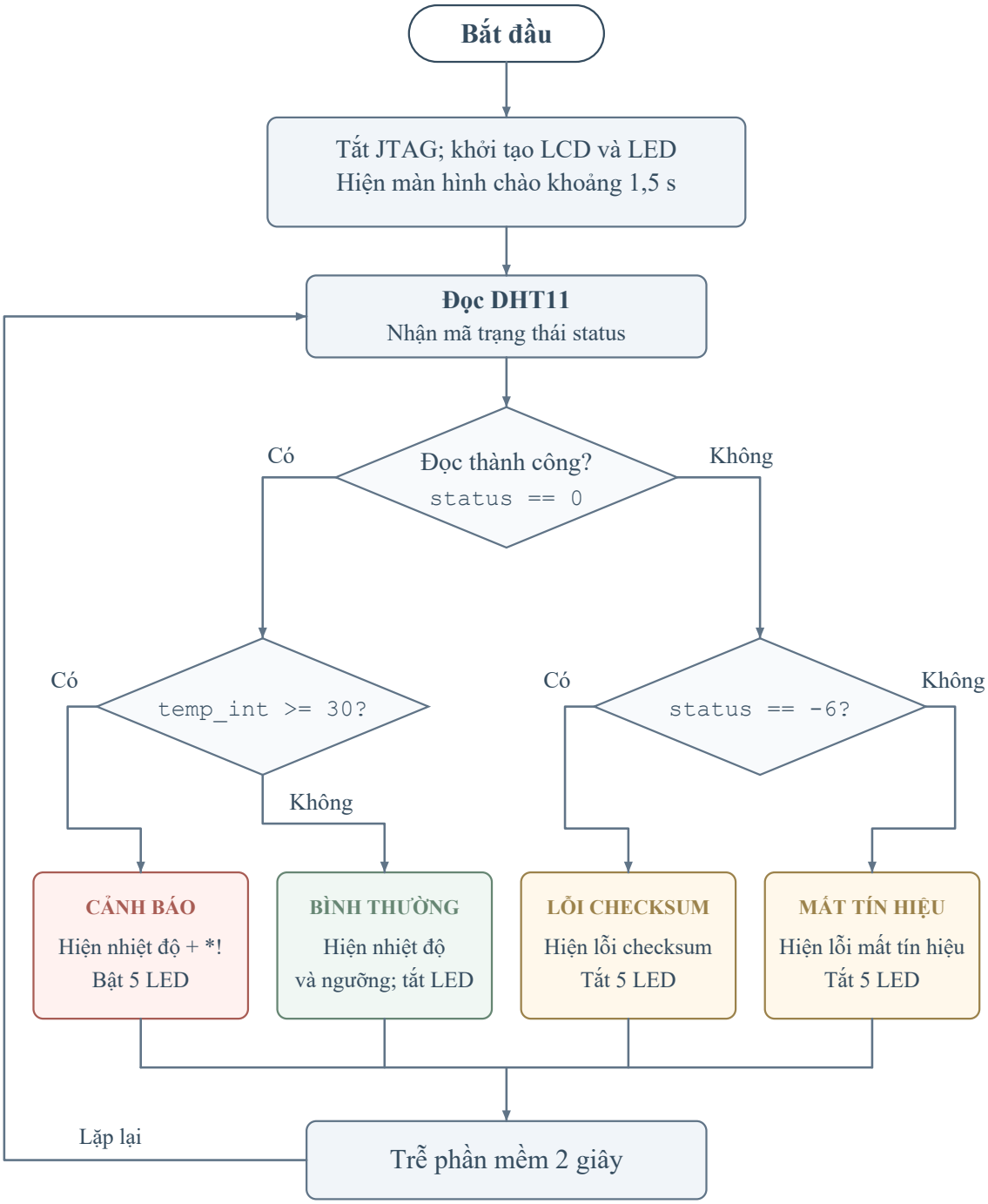
1. **Khởi tạo hệ thống.** Ghi hai lần bit JTD trong `MCUCSR` để tắt JTAG và giải phóng PC4, PC5. Khởi tạo LCD bằng `LCD4_INIT`; đặt PD0–PD4 làm đầu ra (`DDRD |= 0x1F`) và xuất mức cao để tắt LED (`PORTD |= 0x1F`). LCD hiện `DA2: DO NHIET DO` và `DHT11 SENSOR` khoảng 1,5 giây, sau đó được xóa.



Hình 3.3: Mô hình 3D PCB do nhóm thiết kế trên Altium Designer

2. **Đọc cảm biến.** Gọi `DHT11_Read` để nhận dữ liệu nhiệt độ cùng mã trả về `status`. Chương trình chỉ so sánh ngưỡng và hiển thị nhiệt độ khi `status == 0`.
3. **Xử lý dữ liệu hợp lệ.** Dòng 1 LCD hiển thị nhiệt độ theo định dạng `Temp: XX.Y °C`. Dòng 2 hiển thị ngưỡng; các khoảng trắng cuối chuỗi được dùng để ghi đè nội dung cũ trong 16 vị trí:
   - Nếu `temp_int >= 30`: hiện `HIGH Th: 30°C *!`; dùng `PORTD &= 0xE0` kéo PD0–PD4 xuống mức thấp, bật 5 LED.
   - Nếu `temp_int < 30`: hiện `HIGH Th: 30°C`, xóa dấu `*!`; dùng `PORTD |= 0x1F` tắt 5 LED.
4. **Xử lý lỗi.** Không dùng lại dữ liệu cũ để so sánh ngưỡng. Cả hai nhánh lỗi đều tắt 5 LED bằng `PORTD |= 0x1F`:
   - `status == -6`: hiện `DHT11: Sum Error` và `Checksum Invalid` trên hai dòng LCD.
   - `status` từ -1 đến -5: hiện `DHT11: No Signal` và `Check Pin PA1`, tương ứng với timeout khi chờ tín hiệu.

   Việc tắt LED khi lỗi tránh cảnh báo dựa trên dữ liệu cũ, nhưng cũng làm hệ thống không cảnh báo được nếu môi trường đang quá nhiệt; hạn chế này được nêu ở mục 5.2.
5. **Lặp lại.** Thực hiện `_delay_ms(2000)`, rồi quay lại bước đọc cảm biến. Thời gian giữa hai lượt đọc còn bao gồm giao tiếp và cập nhật LCD, như phân tích ở mục 3.2.4.



Hình 3.4: Lưu đồ thuật toán đo nhiệt độ, cảnh báo và xử lý lỗi

3.2.2 Thuật toán giao tiếp và giải mã cảm biến DHT11

Hàm `DHT11_Read(uint8_t *temp_int, uint8_t *temp_dec, uint8_t *humi_int)` được viết trực tiếp bằng thao tác thanh ghi để tạo và lấy mẫu xung theo các mốc thời gian của giao thức, kèm cơ chế timeout chống treo vi điều khiển.

Cơ chế chống treo bằng timeout

Nếu chân DATA bị hở mạch hoặc cảm biến hỏng, các vòng chờ sườn xung như `while(DHT_PIN_R & (1 << DHT_BIT))` sẽ lặp vô hạn và vi điều khiển bị treo. Vì vậy mỗi vòng chờ được gắn một biến đếm giảm dần `timeout = 10000`; khi biến đếm về 0 mà trạng thái chân chưa đổi, hàm thoát và trả về mã lỗi âm ứng với vòng chờ đó.

Thời gian timeout được xác định từ mã máy do `avr-gcc` sinh ra với tùy chọn `-Os` (xem bằng `avr-objdump -d`). Mỗi lần lặp của bốn vòng chờ đầu gồm lệnh kiểm tra bit `sbic`/`sbis` (1 chu kỳ), `rjmp` (2), `sbiw` (2) và `brne` (2), tổng cộng 7 chu kỳ. Ở vòng chờ thứ năm, lệnh `sbis` nhảy qua `rjmp` (2 chu kỳ) nên mỗi lần lặp tốn 6 chu kỳ. Kết quả ở 8 MHz được tổng hợp trong Bảng 3.2.

Bảng 3.2: Thời gian timeout của các vòng chờ trong `DHT11_Read` ở 8 MHz

Mã lỗi	Vòng chờ	Chu kỳ/lần lặp	Timeout
−1	DHT11 kéo đường DATA xuống mức thấp sau xung khởi động	7	8,75 ms
−2	DHT11 kéo đường DATA lên mức cao (tín hiệu phản hồi)	7	8,75 ms
−3	DHT11 kéo đường DATA xuống mức thấp trước bit đầu tiên	7	8,75 ms
−4	Kết thúc mức thấp 50 μs ở đầu mỗi bit	7	8,75 ms
−5	Kết thúc mức cao của bit ‘1’	6	7,5 ms

Khi rút dây DATA, điện trở kéo lên giữ PA1 ở mức cao, vòng chờ đầu tiên hết hạn và hàm trả về −1 sau khoảng 20 ms (xung khởi động) + 8,75 ms ≈ 29 ms, không đáng kể so với chu kỳ 2 giây. Các giá trị trên tỷ lệ nghịch với tần số thực của dao động RC nội. Cũng từ mã máy, `_delay_us(38)` được biên dịch thành vòng trễ 304 chu kỳ, đúng 38 μs ở 8 MHz.

Giải mã bit dữ liệu

Bit '0' có mức cao kéo dài 26–28 μs, bit '1' khoảng 70 μs (mục 2.3.2). Thời điểm lấy mẫu 38 μs nằm giữa hai giá trị này: nếu PA1 còn ở mức cao thì bit là '1', nếu đã về mức thấp thì bit là '0'. Sau xung khởi động và tín hiệu phản hồi, vòng lặp lồng nhau quét 5 byte (40 bit); mỗi bit được giải mã theo các bước:

1. Đợi hết mức thấp 50 μs ở đầu bit đang truyền.
2. Chờ 38 μs (`_delay_us(38)`).
3. Đọc thanh ghi `PINA`: nếu PA1 đang ở mức cao thì bit nhận được là '1' và được ghi vào byte tương ứng:

```
data[i] |= (1 << j);
```

Sau đó chờ mức cao kết thúc rồi mới đọc bit tiếp theo.

Kiểm tra checksum và mã trả về

Cuối hàm `DHT11_Read`, tổng của 4 byte đầu được so sánh với byte thứ 5 (byte checksum):

```
if ((uint8_t)(data[0] + data[1] + data[2] + data[3]) != data[4]) {
    return -6; // Lỗi sai Checksum
}
```

Nếu khớp, các con trỏ nhận giá trị tương ứng và hàm trả về 0. Như vậy hàm có ba nhóm mã trả về: 0 khi đọc thành công; −1 đến −5 khi hết thời gian ở một trong năm vòng chờ của Bảng 3.2; −6 khi sai checksum. Cách `main()` xử lý từng nhóm mã được trình bày ở bước 4 của mục 3.2.1.

3.2.3 Xây dựng thư viện điều khiển màn hình LCD 1602

Thư viện `hunget_lcd.h` điều khiển LCD ở chế độ 4 bit, được cấu hình theo cách nối chân của kit và kiểm tra trên phần cứng. Các mục dưới đây trình bày cách truyền dữ liệu, tạo xung Enable và giữ nguyên các bit không dùng cho LCD.

Hàm tạo xung chốt dữ liệu (LCD_Enable)

Theo datasheet HD44780, dữ liệu trên bus 4 bit được LCD chốt tại sườn xuống của chân Enable (PD7). Ở 8 MHz, mỗi chu kỳ clock chỉ dài 0,125 μs, nên hàm tạo xung phải chèn trễ để bảo đảm độ rộng xung Enable và thời gian LCD xử lý lệnh:

```
static inline void LCD_Enable(void) {
    LCD_CTRL_PORT |= (1 << LCD_EN);
    _delay_us(5);
    LCD_CTRL_PORT &= ~(1 << LCD_EN);
    _delay_us(100);
}
```

Hàm truyền 4 bit dữ liệu (LCD_Send4Bit)

LCD chỉ dùng 4 chân cao của PORT C (PC4–PC7), nên khi ghi dữ liệu không được làm thay đổi 4 chân thấp (PC0–PC3). Chương trình dùng phép AND/OR với mặt nạ bit:

```
static inline void LCD_Send4Bit(unsigned char Data) {
    unsigned char current_port_c = LCD_DATA_PORT & 0x0F;
    LCD_DATA_PORT = current_port_c | (Data & 0xF0);
    LCD_Enable();
}
```

Trình tự khởi tạo chế độ 4-bit (LCD4_INIT)

Khi mới cấp điện, HD44780 có thể chưa ở chế độ giao tiếp mong muốn. Sau khi chờ 50 ms cho LCD ổn định, chương trình gửi mã `0x30` ba lần (cách nhau 5 ms, 2 ms và 2 ms) để đưa HD44780 về trạng thái xác định, sau đó chuyển sang chế độ 4 bit (`0x20`) và cấu hình 2 dòng, ma trận 5 × 8 (`0x28`).

3.2.4 Thuật toán so sánh ngưỡng cảnh báo và điều khiển LED

Ngưỡng cảnh báo được cố định ở 30 °C (`HIGH Th: 30°C`). Đây là mốc kiểm thử để quan sát sự chuyển trạng thái của mô hình trong điều kiện phòng và gia nhiệt nhẹ, không phải ngưỡng an toàn được suy ra từ yêu cầu của một ứng dụng cụ thể. Với dữ liệu nhiệt độ không âm theo định dạng đang dùng, phép so sánh `temp_int >= 30` cũng kích hoạt cảnh báo tại đúng 30 °C.

Hệ thống chưa có vùng trễ (hysteresis): cùng một ngưỡng được dùng để bật và tắt

cảnh báo. Nếu số đo dao động quanh 30 °C, LED có thể đổi trạng thái qua các lần đọc. Đây là hạn chế cần khảo sát ở các điểm sát ngưỡng.

Khi điều khiển LED trên PORT D cần lưu ý rằng PD5, PD6, PD7 đang dùng cho các tín hiệu RW, RS, EN của LCD. Nếu ghi trực tiếp một giá trị vào cả thanh ghi `PORTD`, các tín hiệu này sẽ bị ghi đè và LCD có thể nhận sai lệnh hoặc dữ liệu. Vì vậy chương trình dùng mặt nạ bit:

- Khi `temp_int >= 30`: `PORTD &= 0xE0` (mặt nạ `1110 0000`) đưa PD0–PD4 về 0 để bật LED, giữ nguyên PD5–PD7.
- Khi `temp_int < 30`: `PORTD |= 0x1F` (mặt nạ `0001 1111`) đưa PD0–PD4 lên 1 để tắt LED, giữ nguyên PD5–PD7.

Nhờ mặt nạ bit, khối LED và khối LCD cập nhật các nhóm chân riêng trên cùng PORT D. Chuỗi cảnh báo `HIGH Th: 30°C *!` trên dòng 2 dài đúng 16 ký tự, vừa một dòng của LCD 1602. Sau mỗi vòng lặp, chương trình thực hiện `_delay_ms(2000)`. Chu kỳ giữa hai lần bắt đầu đọc liên tiếp còn bao gồm thời gian giao tiếp DHT11 và cập nhật LCD:

$$T_{\text{chu kỳ}} \approx 2{,}0\,\text{s} + T_{\text{đọc}} + T_{\text{LCD}} + T_{\text{xử lý}}. \tag{3.1}$$

Do đó chu kỳ thực tế lớn hơn 2 giây ở tần số danh định, không phải bộ định thời lấy mẫu chính xác 2 giây. Các khoảng trễ phụ thuộc tần số RC nội thực tế và cấu hình `F_CPU` [7].

3.3 Quy trình biên dịch và nạp firmware

3.3.1 Quy trình tự động hóa với Makefile

Mã nguồn được biên dịch tự động bằng GNU Make và trình biên dịch `avr-gcc` [7]. Tệp `Makefile` gồm các quy tắc:

- Cờ biên dịch: `-Wall -Os -mmcu=atmega16 -DF_CPU=8000000UL` (tối ưu mã máy `-Os`, tần số 8 MHz).
- Quy tắc tạo `tempsen.elf` từ tệp nguồn `tempsen.c` và tệp tiêu đề `hunget_lcd.h`.
- Quy tắc tạo `tempsen.hex`: lệnh `avr-objcopy -O ihex -R .eeprom` chuyển `tempsen.elf` thành tệp Intel HEX để nạp vào bộ nhớ Flash.
- Lệnh `avr-size tempsen.elf` báo dung lượng bộ nhớ mà chương trình sử dụng; kết quả ở Bảng 3.3.

Makefile khai báo `LFUSE = 0xE4` và `HFUSE = 0xD9`; lệnh `make fuse` ghi các giá trị này qua avrdude, chọn dao động RC nội 8 MHz và tắt JTAG ở mức fuse (mục 2.1.3).

Lần nạp ghi nhận trong báo cáo được thực hiện bằng ProgISP với fuse `0x99E4` (mục 3.3.2). Việc biên dịch với `F_CPU` không tự thay đổi fuse phần cứng.

Bảng 3.3: Dung lượng bộ nhớ của chương trình (`avr-size`, avr-gcc 16.1.0, `-Os`)

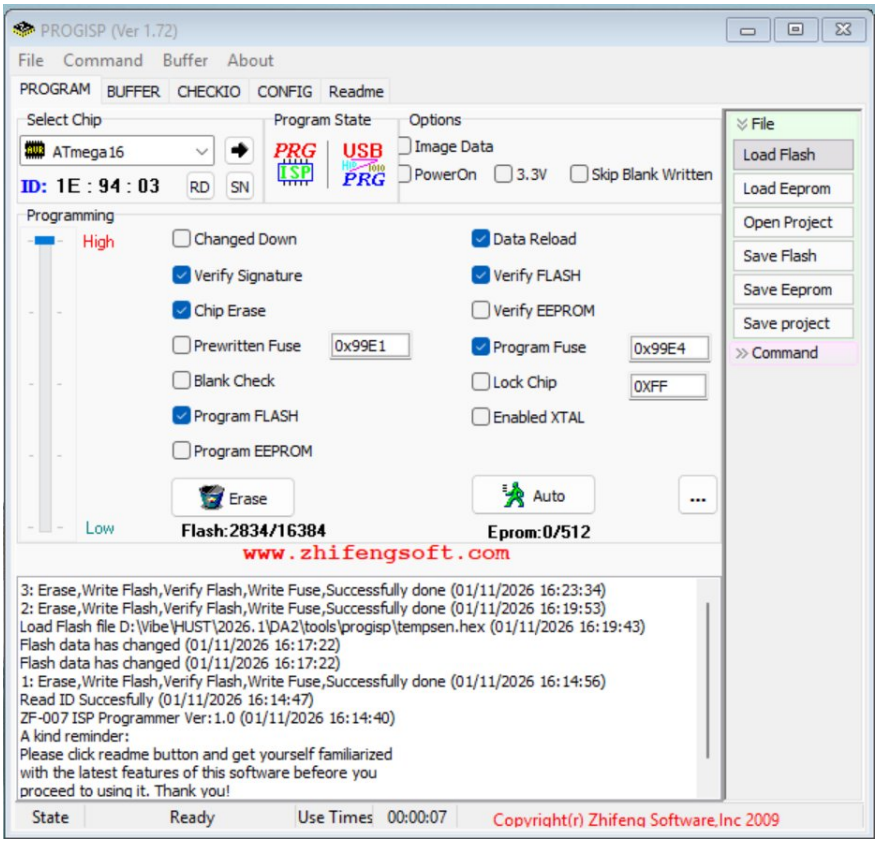
Vùng nhớ	Thành phần	Sử dụng	Dung lượng chip	Tỷ lệ
Flash	`.text` (2674 B) + `.data` (156 B)	2830 B	16384 B	17,3 %
SRAM	`.data` (156 B) + `.bss` (0 B), chưa tính ngăn xếp	156 B	1024 B	15,2 %
EEPROM	Không sử dụng	0 B	512 B	0 %

Toàn bộ 156 B của vùng `.data` là các chuỗi hằng hiển thị trên LCD (cùng chuỗi định dạng `"%05u"` của thư viện LCD); chúng được chép từ Flash sang SRAM khi khởi động. Biến cục bộ như `buffer[32]` trong `main()` và vùng nhớ tạm của `sprintf` nằm trên ngăn xếp nên không có trong số liệu này. Chương trình còn nhiều dư địa bộ nhớ cho các hướng mở rộng ở mục 5.3.

3.3.2 Quy trình nạp chương trình bằng phần mềm ProgISP

Các bước nạp chương trình vào ATmega16 được trình bày dưới đây; giao diện tham khảo ở Hình 3.5.

1. Kết nối USBasp giữa máy tính và cổng ISP trên kit.
2. Mở ProgISP 1.72.
3. Tại *Select Chip*, chọn `ATMEGA16`.
4. Nhấn RD (Read) để kiểm tra kết nối; mã nhận diện dự kiến là `1E : 94 : 03`.
5. Bật tùy chọn *Program Fuse* và nhập giá trị fuse. Ở lần nạp trong Hình 3.5, giá trị là `0x99E4` (High Fuse `0x99`, Low Fuse `0xE4`): Low Fuse chọn dao động RC nội 8 MHz, High Fuse giữ giá trị xuất xưởng nên JTAG được tắt bằng phần mềm (mục 2.1.3). Để tắt JTAG cả ở mức fuse, nhập `0xD9E4`.
6. Nhấn *Load Flash*, chọn tệp `src/tempsen.hex`.
7. Nhấn *Auto* để xóa chip, ghi và kiểm tra Flash, ghi fuse. Khi báo *Successfully done*, vi điều khiển khởi động lại và chạy chương trình.



Hình 3.5: Giao diện ProgISP khi nạp firmware cho mô hình (fuse `0x99E4`)

CHƯƠNG 4

KẾT QUẢ THỰC NGHIỆM

4.1 Kết quả thực nghiệm trên phần cứng

Hệ thống được lắp ráp, nạp chương trình và kiểm thử trên kit AVR tại phòng thí nghiệm Trường Điện – Điện tử, Đại học Bách khoa Hà Nội. Các trạng thái ghi nhận trong quá trình thử nghiệm được trình bày dưới đây; phạm vi và giới hạn của phép kiểm chứng ở mục 4.2.2.

4.1.1 Giai đoạn khởi động

Khi cấp nguồn 5 V từ cổng USB, vi điều khiển khởi động, tắt JTAG, khởi tạo LCD và các chân LED. LCD sau đó hiển thị màn hình chào:

Dòng 1	`DA2: DO NHIET DO`
Dòng 2	`DHT11 SENSOR`

Màn hình chào được giữ khoảng 1,5 giây theo trễ phần mềm. Trong giai đoạn này, 5 LED cảnh báo tắt do PD0–PD4 được đặt mức 1 khi khởi tạo (`PORTD |= 0x1F`). Sau đó chương trình gửi lệnh xóa màn hình (`0x01`) trước khi bắt đầu vòng lặp đo.

4.1.2 Trạng thái bình thường ($T < 30\,^{\circ}C$)

Ở trạng thái không cảnh báo, LCD hiển thị nhiệt độ 26,5 °C, nhỏ hơn ngưỡng 30 °C:

- DHT11 gửi gói tin 40 bit về vi điều khiển; checksum khớp nên dữ liệu được chấp nhận.
- LCD 1602 hiển thị:

Dòng 1	`Temp: 26.5 °C`
Dòng 2	`HIGH Th: 30°C`

- Vì nhiệt độ dưới ngưỡng, dòng 2 chỉ hiển thị ngưỡng, không có ký hiệu cảnh báo.
- 5 LED cảnh báo trên PD0–PD4 giữ mức 1 nên tắt.

4.1.3 *Trạng thái cảnh báo quá nhiệt ($T \geq 30\,^{\circ}C$)*

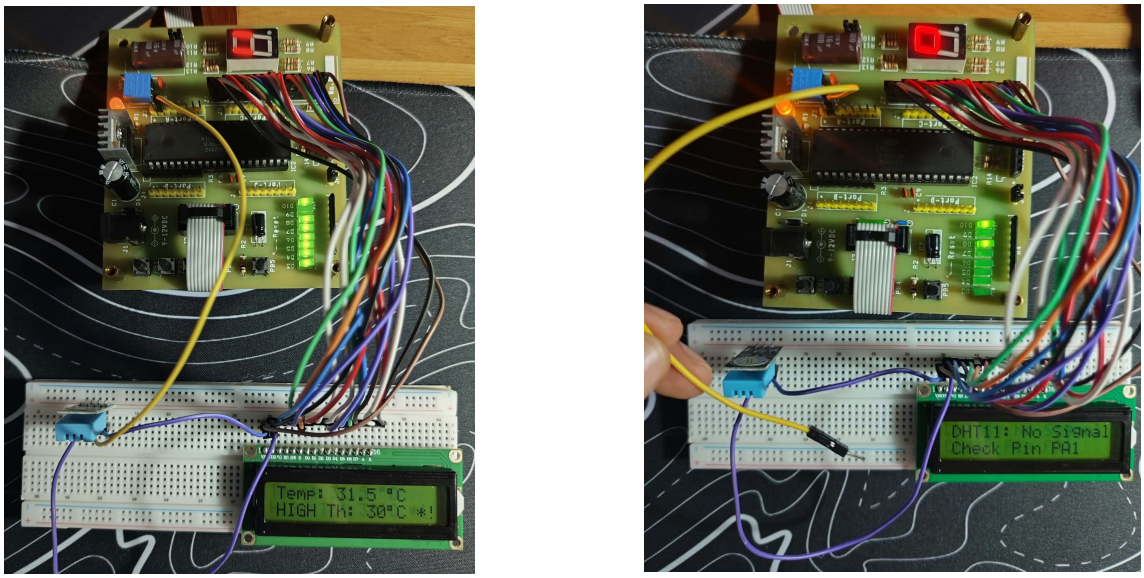
Để tạo trạng thái quá nhiệt, nhóm đưa một nguồn hơi ấm nhẹ lại gần bề mặt DHT11. Khi nhiệt độ nhận được đạt hoặc vượt 30 °C:

- Vi điều khiển phát hiện điều kiện `temp_int >= 30` ở lần đọc hợp lệ tiếp theo (mỗi vòng lặp gồm trễ 2 giây cộng thời gian đọc và hiển thị).
- LCD hiển thị:

Dòng 1	`Temp: 31.5 °C`
Dòng 2	`HIGH Th: 30°C *!`

  Ký hiệu `*!` xuất hiện ở cuối dòng 2.

- Đồng thời, lệnh `PORTD &= 0xE0` kéo PD0–PD4 xuống mức 0, 5 LED cảnh báo sáng.
- Khi bỏ nguồn nhiệt và nhiệt độ giảm xuống dưới 30 °C, ký hiệu `*!` biến mất và 5 LED tắt mà không cần reset.



(a) Cảnh báo quá nhiệt (31,5 °C) **(b)** Mất tín hiệu cảm biến

Hình 4.1: Hình ảnh thực tế hệ thống trong các kịch bản thử nghiệm

Ghi chú: Dãy LED trên kit gồm 8 LED nối PD0–PD7 (từ D3 ở dưới lên D10 ở trên). Ba LED trên cùng (PD5–PD7) dùng chung đường RW, RS, EN của LCD nên sáng theo mức logic của các tín hiệu này, không thuộc khối cảnh báo. Ở (a), 5 LED cảnh báo PD0–PD4 sáng; ở (b), 5 LED này tắt.

4.1.4 Kiểm tra xử lý lỗi cảm biến

Nhóm kiểm tra hai tình huống lỗi:

- Mất tín hiệu cảm biến (thử trên phần cứng): nhóm rút dây DATA nối với PA1. Vòng chờ đầu tiên của `DHT11_Read` hết hạn và hàm trả về -1 sau khoảng 29 ms (Bảng 3.2); vi điều khiển không bị treo. LCD hiển thị thông báo lỗi (Hình 4.1b):

Dòng 1	`DHT11: No Signal`
Dòng 2	`Check Pin PA1`

- Sai checksum (phân tích mã nguồn): nhóm chưa tạo được nhiễu trên đường truyền để gây lỗi checksum, nên nhánh `status == -6` mới được kiểm tra bằng cách đọc mã nguồn. Theo mã nguồn, LCD sẽ hiển thị:

Dòng 1	`DHT11: Sum Error`
Dòng 2	`Checksum Invalid`

- Trong cả hai trường hợp, 5 LED cảnh báo được tắt (`PORTD |= 0x1F`) như phân tích ở mục 3.2.1.

Bảng 4.1: Tổng hợp các kịch bản kiểm tra và kết quả

Kịch bản kiểm tra	**Phương pháp xác minh**	**Giá trị đo**	**Nội dung hiển thị trên LCD 1602**	**Trạng thái LED**
Khởi động hệ thống	Thực nghiệm trên kit	–	Dòng 1: `DA2: DO NHIET DO` Dòng 2: `DHT11 SENSOR`	Tắt toàn bộ
Nhiệt độ bình thường	Thực nghiệm trên kit	26,5 °C	Dòng 1: `Temp: 26.5 °C` Dòng 2: `HIGH Th: 30°C`	Tắt toàn bộ
Nhiệt độ vượt ngưỡng	Thực nghiệm trên kit	31,5 °C	Dòng 1: `Temp: 31.5 °C` Dòng 2: `HIGH Th: 30°C *!`	Sáng cả 5 LED
Mất tín hiệu cảm biến	Thực nghiệm trên kit (rút dây PA1)	–	Dòng 1: `DHT11: No Signal` Dòng 2: `Check Pin PA1`	Tắt toàn bộ
Sai checksum	Phân tích mã nguồn	–	Dòng 1: `DHT11: Sum Error` Dòng 2: `Checksum Invalid`	Tắt toàn bộ

4.2 Đánh giá kết quả và phạm vi kiểm chứng

4.2.1 Kết quả chức năng đã ghi nhận

Các kết quả ở mục 4.1 cho thấy chương trình hiển thị được nhiệt độ, báo vượt ngưỡng bằng LCD và LED, và hiển thị thông báo khi mất tín hiệu cảm biến. Bảng 4.1 phân biệt các kịch bản thử trên phần cứng với nhánh checksum mới được kiểm tra qua mã nguồn. Điều kiện `temp_int >= 30` bao gồm cả trường hợp đúng ngưỡng, nhưng báo cáo chưa có kết quả thử riêng tại 30 °C.

Hình 4.1a ghi nhận LCD hiển thị 31,5 °C cùng ký hiệu `*!` và 5 LED cảnh báo sáng. Vì chương trình in trực tiếp T_{int} và T_{dec} nhận từ cảm biến, phần thập phân `.5` trên LCD chính là giá trị Byte 3 của gói tin. Như vậy module đang dùng trả về byte thập phân khác 0, khác với độ phân giải 1 °C ghi trong tài liệu tham chiếu [2]. Các giá trị này phản ánh dữ liệu cảm biến gửi về; độ chính xác so với nhiệt độ thực của môi trường cần được đối chiếu thêm với nhiệt kế tham chiếu.

Checksum chỉ xác nhận gói tin được truyền đúng, không xác nhận độ chính xác của phép đo. Ngoài ra, khi có lỗi chương trình tắt 5 LED, nên LED tắt không đủ để kết luận môi trường đang dưới ngưỡng.

4.2.2 Giới hạn của kết quả thực nghiệm

Báo cáo hiện chưa có các dữ liệu sau để đánh giá định lượng:

- Các cặp số đo đồng thời giữa hệ thống và nhiệt kế tham chiếu tại nhiều mức nhiệt, nên chưa xác định được sai lệch trung bình hoặc sai lệch lớn nhất.
- Nhật ký chạy liên tục gồm tổng số lần đọc, số lần hợp lệ, số lỗi timeout và checksum, nên chưa tính được tỷ lệ đọc thành công hoặc độ ổn định dài hạn.
- Thời gian từ khi số đo chạm ngưỡng đến khi LED đổi trạng thái; trễ 2 giây trong mã nguồn không thay thế được phép đo độ trễ toàn hệ thống.
- Thử lặp lại ở dưới ngưỡng, đúng ngưỡng và trên ngưỡng, cũng như rút rồi nối lại cảm biến khi hệ thống đang cảnh báo.

Kết quả hiện có xác nhận các chức năng cơ bản, nhưng chưa xác nhận độ chính xác đo hoặc độ tin cậy dài hạn.

CHƯƠNG 5

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận

Đề tài đã xây dựng mô hình đo nhiệt độ và cảnh báo ngưỡng dùng ATmega16, DHT11 và LCD 1602. Các kết quả chính gồm:

- Vẽ lại sơ đồ nguyên lý từ tài liệu giảng viên cung cấp và thiết kế PCB trên Altium Designer; xác định cách nối cảm biến ngoài kit và các nhóm chân dùng cho LCD, LED.
- Viết chương trình C đọc gói dữ liệu 40 bit từ DHT11, kiểm tra checksum, giới hạn mỗi vòng chờ bằng timeout 7,5–8,75 ms và hiển thị thông báo lỗi theo từng loại. Chương trình chiếm 2830 B (17,3 %) bộ nhớ Flash.
- Cấu hình thư viện LCD giao tiếp 4 bit và dùng mặt nạ bit để LCD và LED dùng chung PORT D mà không ảnh hưởng nhau.
- Cấu hình dao động RC nội 8 MHz bằng fuse (Low Fuse `0xE4`), thống nhất với giá trị `F_CPU`; tắt JTAG bằng lệnh ghi `JTD` để dùng PC4, PC5 cho LCD.
- Ghi nhận trên phần cứng các trạng thái hiển thị nhiệt độ, cảnh báo vượt ngưỡng và báo mất tín hiệu cảm biến (Chương 4). Nhánh lỗi checksum hiện mới được kiểm tra bằng phân tích mã nguồn.

Mô hình thực hiện đúng các chức năng đề ra trong phạm vi thử nghiệm đã trình bày; các hạn chế còn lại được tổng hợp ở mục 5.2.

5.2 Hạn chế của đề tài

Các hạn chế hiện tại của mô hình gồm:

- DHT11 có dải đo hẹp (0–50 °C), sai số ± 2 °C và chu kỳ đọc khoảng 2 giây, nên không phù hợp với ứng dụng cần độ chính xác cao hoặc đáp ứng nhanh.
- Chưa có phép đo đối chiếu với nhiệt kế tham chiếu, nhật ký chạy dài hạn và phép

đo độ trễ cảnh báo, nên độ chính xác và độ ổn định mới được đánh giá ở mức chức năng (mục 4.2.2).

- Chưa có vùng trễ (hysteresis), nên cảnh báo có thể bật/tắt liên tục khi số đo dao động quanh ngưỡng.
- Ngưỡng 30 °C được cố định trong mã nguồn; muốn đổi ngưỡng phải biên dịch và nạp lại chương trình.
- Khi mất cảm biến, chương trình tắt toàn bộ LED để tránh cảnh báo dựa trên dữ liệu cũ; nhược điểm là hệ thống không còn cảnh báo nếu môi trường vẫn đang quá nhiệt.
- Hệ thống chưa lưu lịch sử nhiệt độ và chưa truyền dữ liệu từ xa.

5.3 Hướng phát triển

Từ mô hình hiện có, hệ thống có thể phát triển theo các hướng sau:

1. Hoàn thiện kiểm chứng: ghi dữ liệu đo, đối chiếu với nhiệt kế tham chiếu, thống kê lỗi khi chạy liên tục và đo độ trễ cảnh báo. Bổ sung hysteresis với ngưỡng bật và tắt riêng, chọn theo yêu cầu ứng dụng và độ dao động của số đo.
2. Nâng cấp cảm biến: thay DHT11 bằng cảm biến số chính xác hơn tùy yêu cầu ứng dụng, ví dụ DHT22 hoặc các cảm biến I2C như Sensirion SHT30/SHT35, Bosch BME280 (đo thêm áp suất khí quyển).
3. Cài đặt ngưỡng: dùng các nút bấm trên PORT B của kit hoặc bàn phím ma trận 4×4 để chỉnh ngưỡng, lưu giá trị vào EEPROM của ATmega16 để giữ lại khi mất điện; bổ sung còi báo.
4. Phân biệt lỗi cảm biến: cho LED nhấp nháy (ví dụ 2 Hz) hoặc còi kêu ngắt quãng khi mất cảm biến, để người vận hành phân biệt ba trạng thái: bình thường (LED tắt), quá nhiệt (LED sáng liên tục) và lỗi cảm biến (LED nhấp nháy).
5. Giám sát từ xa: dùng USART của ATmega16 giao tiếp với module Wi-Fi (ESP8266/ESP32), Bluetooth (HC-05) hoặc LoRa để gửi dữ liệu lên máy chủ qua MQTT/HTTP, cho phép theo dõi và nhận cảnh báo trên điện thoại. Khi dùng USART phần cứng cần chuyển LED khỏi PD0/RXD và PD1/TXD, đồng thời kiểm tra mức điện áp giao tiếp của module (ESP8266/ESP32 dùng mức 3,3 V).

TÀI LIỆU THAM KHẢO

[1] Atmel Corporation, “ATmega16/ATmega16L Datasheet”, Rev. 2466T–AVR–07/10, 2010. https://ww1.microchip.com/downloads/en/DeviceDoc/doc2466.pdf, truy cập ngày 01/11/2026.

[2] Aosong Electronics, “DHT11 Digital Temperature and Humidity Sensor” (bản tài liệu tiếng Trung, 6 trang, không ghi năm), mục 1, 4 và 6. Bản lưu: https://cdn-shop.adafruit.com/datasheets/DHT11-chinese.pdf, truy cập ngày 01/11/2026.

[3] Texas Instruments, “LM35 Precision Centigrade Temperature Sensors”, SNIS159H, 08/1999, sửa đổi 12/2017. https://www.ti.com/lit/ds/symlink/lm35.pdf, truy cập ngày 02/11/2026.

[4] Dallas Semiconductor (nay thuộc Analog Devices), “DS18B20 Programmable Resolution 1-Wire Digital Thermometer”. Bản lưu: https://cdn.sparkfun.com/datasheets/Sensors/Temp/DS18B20.pdf, truy cập ngày 02/11/2026.

[5] Hitachi, “HD44780U (LCD-II): Dot Matrix Liquid Crystal Display Controller/Driver”, ADE-207-272(Z), Rev. 0.0, 09/1999. Bản lưu: https://cdn.sparkfun.com/assets/9/5/f/7/b/HD44780.pdf, truy cập ngày 01/11/2026.

[6] “Sơ đồ mạch Kit”, tài liệu nội bộ do giảng viên cung cấp phục vụ học phần Đồ án Thiết kế II; tệp lưu trong hồ sơ đồ án tại `docs/ref`. Không ghi tác giả và năm xuất bản. Nhóm sử dụng làm cơ sở vẽ lại sơ đồ và thiết kế PCB.

[7] AVR-LibC Project, “AVR-LibC User Manual: delay.h File Reference”. https://avrdudes.github.io/avr-libc/avr-libc-user-manual/delay_8h.html, truy cập ngày 01/11/2026.

PHỤ LỤC: TOÀN BỘ MÃ NGUỒN CHƯƠNG TRÌNH

Phụ lục A: File mã nguồn chính `tempsen.c`

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>
#include "hunget_lcd.h" // Thư viện điều khiển LCD 16x2 chế độ 4 bit

// ======================================================
// CẤU HÌNH CHÂN DHT11 NỐI VÀO CHÂN PA1 TRÊN PORT A
// ======================================================
#define DHT_DDR   DDRA
#define DHT_PORT  PORTA
#define DHT_PIN_R PINA
#define DHT_BIT   PA1

// Hàm đọc dữ liệu từ cảm biến số DHT11
// Trả về 0 nếu thành công, < 0 nếu lỗi/timeout
int8_t DHT11_Read(uint8_t *temp_int, uint8_t *temp_dec, uint8_t *humi_int) {
    uint8_t data[5] = {0, 0, 0, 0, 0};
    uint16_t timeout;

    // 1. MCU gửi tín hiệu Start (kéo LOW tối thiểu 18ms)
    DHT_DDR |= (1 << DHT_BIT);     // Set PA1 là Output
    DHT_PORT &= ~(1 << DHT_BIT);   // Kéo xuống LOW
    _delay_ms(20);                 // Giữ mức LOW 20ms

    // 2. Nhả bus sang Input và kích hoạt trở kéo lên để chờ DHT11 phản hồi
     ↪  (tránh xung đột bus)
    DHT_DDR &= ~(1 << DHT_BIT);    // Chuyển PA1 thành Input
    DHT_PORT |= (1 << DHT_BIT);    // Bật điện trở kéo lên nội
    _delay_us(30);                 // Chờ 20-40us cho DHT11 bắt đầu phản hồi

    // Đợi DHT11 kéo LOW (khoảng 20-40µs)
    timeout = 10000;
    while (DHT_PIN_R & (1 << DHT_BIT)) {
        if (--timeout == 0) return -1; // Timeout chờ DHT11 phản hồi
    }

    // Đợi DHT11 kéo HIGH (khoảng 80µs)
    timeout = 10000;
    while (!(DHT_PIN_R & (1 << DHT_BIT))) {
        if (--timeout == 0) return -2;
    }

    // Đợi DHT11 kéo LOW chuẩn bị truyền bit đầu tiên (khoảng 80µs)
    timeout = 10000;
    while (DHT_PIN_R & (1 << DHT_BIT)) {
        if (--timeout == 0) return -3;
```

```
    }

    // 3. Đọc 40 bit (5 byte: Humi_int, Humi_dec, Temp_int, Temp_dec, Checksum)
    for (uint8_t i = 0; i < 5; i++) {
        for (int8_t j = 7; j >= 0; j--) {
            // Đợi hết xung LOW 50µs của từng bit
            timeout = 10000;
            while (!(DHT_PIN_R & (1 << DHT_BIT))) {
                if (--timeout == 0) return -4;
            }

            // Đo độ rộng xung HIGH:
            // 26-28µs -> bit 0
            // 70µs    -> bit 1
            // Chờ 38µs rồi đọc trạng thái chân
            _delay_us(38);
            if (DHT_PIN_R & (1 << DHT_BIT)) {
                data[i] |= (1 << j);
                // Đợi xung HIGH kết thúc
                timeout = 10000;
                while (DHT_PIN_R & (1 << DHT_BIT)) {
                    if (--timeout == 0) return -5;
                }
            }
        }
    }

    // 4. Kiểm tra mã kiểm tra lỗi (Checksum 8-bit)
    if ((uint8_t)(data[0] + data[1] + data[2] + data[3]) != data[4]) {
        return -6; // Sai checksum
    }

    *humi_int  = data[0];
    *temp_int  = data[2];
    *temp_dec  = data[3];
    return 0; // Đọc thành công
}

int main(void) {
    // 1. TẮT JTAG (BẮT BUỘC để giải phóng PORT C điều khiển LCD)
    MCUCSR = (1 << JTD);
    MCUCSR = (1 << JTD);

    // 2. KHỞI TẠO LCD
    LCD4_INIT(0, 0);

    // 3. KHỞI TẠO DÃY LED CẢNH BÁO (PD0 - PD4)
    // PD5-PD7 là Output cho LCD (đã set trong thư viện LCD)
    // Set thêm PD0-PD4 là Output cho LED
    DDRD |= 0x1F;  // Set 5 chân thấp là Output
    PORTD |= 0x1F; // Xuất mức 1 để TẮT hết LED ban đầu (Active Low)

    // --- MÀN HÌNH CHÀO MỪNG ---
    LCD4_CUR_GOTO(1, 0);
```

```
LCD4_OUT_STR("DA2: DO NHIET DO");
LCD4_CUR_GOTO(2, 2);
LCD4_OUT_STR("DHT11 SENSOR");
_delay_ms(1500);
LCD4_OUT_CMD(0x01); // Xóa màn hình
_delay_ms(2);

char buffer[32];
uint8_t temp_int = 0;
uint8_t temp_dec = 0;
uint8_t humi_int = 0;

while (1) {
    int8_t status = DHT11_Read(&temp_int, &temp_dec, &humi_int);

    if (status == 0) {
        // --- ĐỌC THÀNH CÔNG TỪ DHT11 ---
        // Dòng 1: "Temp: 31.5 °C   " (đúng 16 ký tự chuẩn)
        LCD4_CUR_GOTO(1, 0);
        sprintf(buffer, "Temp: %u.%u \xDF""C   ", temp_int, temp_dec);
        LCD4_OUT_STR(buffer);

        // Dòng 2: Hiển thị ngưỡng cảnh báo "HIGH Th: 30°C"
        LCD4_CUR_GOTO(2, 0);
        if (temp_int >= 30) {
            // Nhiệt độ vượt ngưỡng 30 độ C: Báo WARNING và BẬT LED (đúng 16
            ↪ ký tự)
            LCD4_OUT_STR("HIGH Th: 30\xDF""C *!");
            PORTD &= 0xE0; // Bật sáng 5 LED cảnh báo (Active LOW)
        } else {
            // Nhiệt độ bình thường < 30 độ C: TẮT LED (đúng 16 ký tự, xóa
            ↪ dấu *!)
            LCD4_OUT_STR("HIGH Th: 30\xDF""C   ");
            PORTD |= 0x1F; // Tắt 5 LED cảnh báo
        }
    } else if (status == -6) {
        // Lỗi kiểm tra toàn vẹn dữ liệu (Checksum Error)
        LCD4_CUR_GOTO(1, 0);
        LCD4_OUT_STR("DHT11: Sum Error");
        LCD4_CUR_GOTO(2, 0);
        LCD4_OUT_STR("Checksum Invalid");
        PORTD |= 0x1F; // Trạng thái xác định: Tắt LED cảnh báo
    } else {
        // Trường hợp mất tín hiệu, chưa cắm chân PA1 hoặc lỏng dây
        LCD4_CUR_GOTO(1, 0);
        LCD4_OUT_STR("DHT11: No Signal");
        LCD4_CUR_GOTO(2, 0);
        LCD4_OUT_STR("Check Pin PA1   ");
        PORTD |= 0x1F; // Trạng thái xác định: Tắt LED cảnh báo
    }

    // Chu kỳ lấy mẫu khuyến nghị theo datasheet của Aosong (2 giây)
    _delay_ms(2000);
}
```

```
    return 0;
}
```

Phụ lục B: Thư viện điều khiển LCD `hunget_lcd.h`

```
#ifndef HUNGET_LCD_H
#define HUNGET_LCD_H

#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>
#include <stdlib.h>

// ========================================================
// CẤU HÌNH CHÂN THEO ĐÚNG SƠ ĐỒ MẠCH KIT
// ========================================================
// 1. CHÂN DỮ LIỆU (DATA) -> PORT C
// Sử dụng 4 bit cao: D4->PC4, D5->PC5, D6->PC6, D7->PC7
#define LCD_DATA_DDR DDRC
#define LCD_DATA_PORT PORTC
#define LCD_DATA_PIN PINC

// 2. CHÂN ĐIỀU KHIỂN (CONTROL) -> PORT D
#define LCD_CTRL_DDR DDRD
#define LCD_CTRL_PORT PORTD

// Định nghĩa lại chính xác theo sơ đồ J7:
#define LCD_RW 5 // Pin 5 LCD nối PD5
#define LCD_RS 6 // Pin 4 LCD nối PD6
#define LCD_EN 7 // Pin 6 LCD nối PD7

// ========================================================
// CÁC HÀM GIAO TIẾP MỨC THẤP
// ========================================================
// Hàm tạo xung Enable trên PORT D
static inline void LCD_Enable(void) {
    LCD_CTRL_PORT |= (1 << LCD_EN); // Bật E (PD7)
    _delay_us(5);
    LCD_CTRL_PORT &= ~(1 << LCD_EN); // Tắt E
    _delay_us(100);
}

// Hàm gửi 4 bit dữ liệu ra PORT C
static inline void LCD_Send4Bit(unsigned char Data) {
    // 1. Gửi 4 bit cao của dữ liệu ra PORT C (vào các chân PC4-PC7)
    // Giữ nguyên trạng thái của 4 chân thấp PC0-PC3 bằng mặt nạ bit
    unsigned char current_port_c = LCD_DATA_PORT & 0x0F;
    LCD_DATA_PORT = current_port_c | (Data & 0xF0);
    // 2. Tạo xung chốt bên PORT D
    LCD_Enable();
}
```

```
// Hàm gửi lệnh (RS=0, RW=0 tại PORT D)
static inline void LCD4_OUT_CMD(unsigned char command) {
    LCD_CTRL_PORT &= ~(1 << LCD_RS);
    LCD_CTRL_PORT &= ~(1 << LCD_RW);
    LCD_Send4Bit(command); // Gửi 4 bit cao (sang PORT C)
    LCD_Send4Bit(command << 4); // Gửi 4 bit thấp (sang PORT C)
    if (command < 4) _delay_ms(2);
    else _delay_us(50);
}

// Hàm gửi dữ liệu (RS=1, RW=0 tại PORT D)
static inline void LCD4_OUT_DATA(unsigned char data) {
    LCD_CTRL_PORT |= (1 << LCD_RS);
    LCD_CTRL_PORT &= ~(1 << LCD_RW);
    LCD_Send4Bit(data);
    LCD_Send4Bit(data << 4);
    _delay_us(50);
}

// ========================================================
// CÁC HÀM NGƯỜI DÙNG (USER FUNCTIONS)
// ========================================================
static inline void LCD4_INIT(unsigned char cursor, unsigned char blink) {
    // Khởi tạo hướng dữ liệu (Output)
    // PC4-PC7 là Output (Data)
    LCD_DATA_DDR |= 0xF0;
    // PD5, PD6, PD7 là Output (Control)
    LCD_CTRL_DDR |= (1<<LCD_RS) | (1<<LCD_RW) | (1<<LCD_EN);
    // Reset mức logic các chân điều khiển về 0
    LCD_CTRL_PORT &= ~((1<<LCD_RS) | (1<<LCD_RW) | (1<<LCD_EN));
    _delay_ms(50); // Chờ LCD ổn định sau khi cấp nguồn

    // Chuỗi Reset (Gửi 0x30 ba lần theo chuẩn HD44780)
    LCD_Send4Bit(0x30); _delay_ms(5);
    LCD_Send4Bit(0x30); _delay_ms(2);
    LCD_Send4Bit(0x30); _delay_ms(2);

    // Chuyển sang chế độ 4-bit
    LCD_Send4Bit(0x20);
    _delay_ms(2);

    // Cấu hình hiển thị
    LCD4_OUT_CMD(0x28); // 4-bit, 2 dòng, font 5x8
    _delay_ms(1);
    LCD4_OUT_CMD(0x08); // Tắt hiển thị
    _delay_ms(1);
    LCD4_OUT_CMD(0x01); // Xóa màn hình
    _delay_ms(3);

    unsigned char cmd = 0x0C; // Mặc định bật màn hình
    if(cursor) cmd |= 0x02;
    if(blink) cmd |= 0x01;
    LCD4_OUT_CMD(cmd);
```

```
    _delay_ms(1);
    LCD4_OUT_CMD(0x06); // Tự động tăng con trỏ
    _delay_ms(1);
}

void LCD4_CUR_GOTO(unsigned char row, unsigned char col) {
    unsigned char address;
    if (row == 1) address = 0x80 + col;
    else address = 0xC0 + col;
    LCD4_OUT_CMD(address);
}

void LCD4_OUT_STR(char *str) {
    while (*str) LCD4_OUT_DATA(*str++);
}

void LCD4_OUT_DEC(unsigned int num, unsigned char len) {
    char buffer[10];
    sprintf(buffer, "%05u", num);
    char *ptr = &buffer[5 - len];
    LCD4_OUT_STR(ptr);
}

void LCD4_DIS_SHIFT(unsigned char direct, unsigned char step) {
    unsigned char i;
    if (direct == 0) // Dịch trái
        for(i=0; i<step; i++) LCD4_OUT_CMD(0x18);
    else // Dịch phải
        for(i=0; i<step; i++) LCD4_OUT_CMD(0x1C);
}

#endif // HUNGET_LCD_H
```